PYTHON ACTIVITY-BASED LEARNING

10 Selected Case Studies — Complete Practical Record

Case Study 1: ATM Transaction Sequence Simulation — Easy

1. Problem Statement

Process a sequence of deposits and withdrawals, maintain a simulated balance, enforce withdrawal limits, reject invalid transactions, and produce a transaction summary.

2. Objectives

- Maintain an account balance using a sequence of transactions.
- Accept valid deposits and withdrawals.
- Reject invalid amounts and withdrawals beyond the allowed limit or balance.
- Display a final transaction summary.

3. Assumptions

- Opening balance is ₹5,000.
- Each transaction is entered as D for deposit or W for withdrawal.
- Withdrawal amount must be positive and cannot exceed ₹3,000 per transaction or the available balance.
- Only valid transaction types are processed.

4. Algorithm

1. Start with balance = 5000 and counters for deposits, withdrawals and rejected transactions.

2. Read the number of transactions.

3. For each transaction, read type and amount.

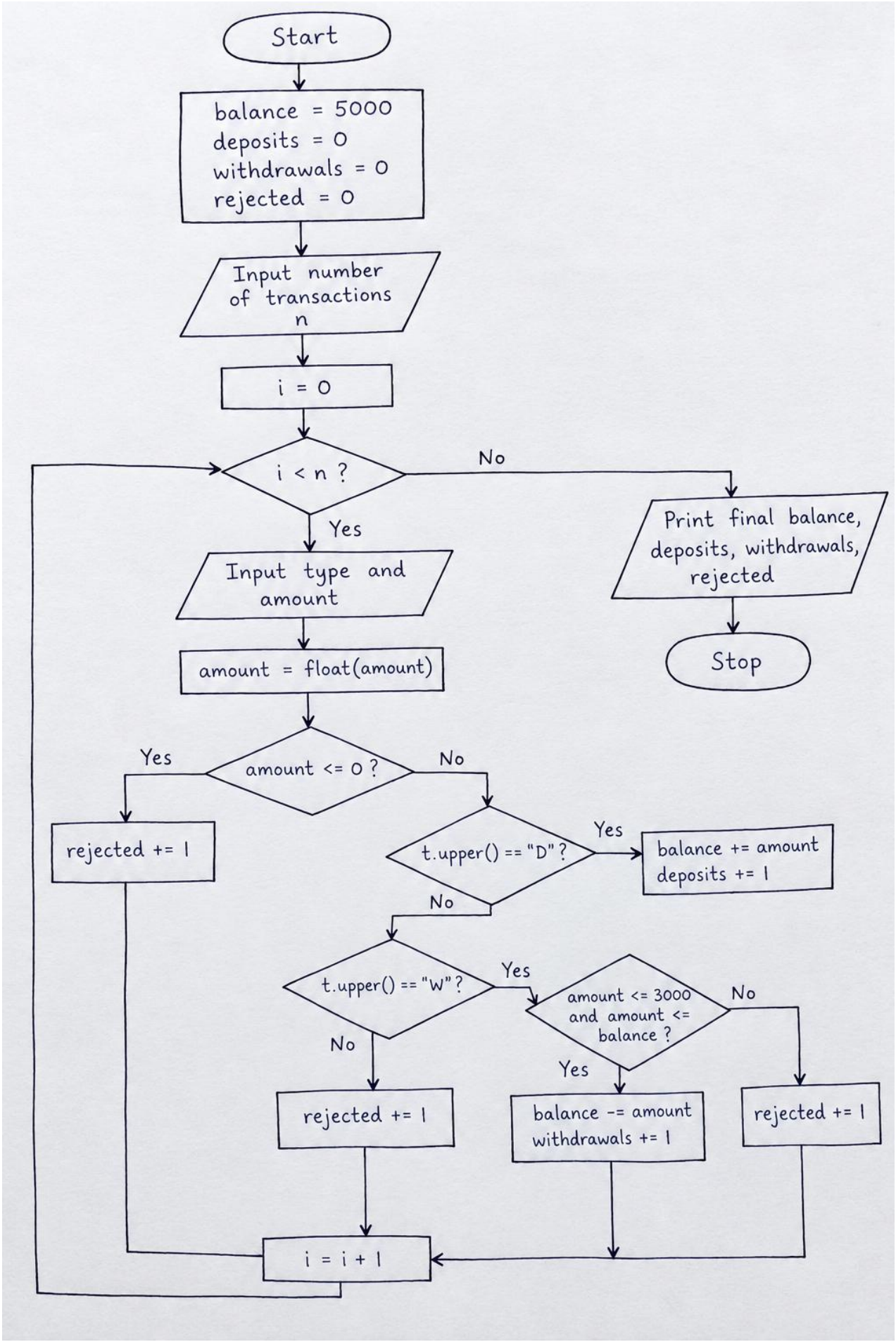
4. If the amount is not positive, reject it.

5. If type is D, add the amount to balance.

6. If type is W, check the ₹3,000 limit and available balance; otherwise reject it.

7. After all transactions, display the final balance and summary.

5. Flowchart


Start
balance = 5000
deposits = 0
withdrawals = 0
rejected = 0
Input number of transactions n
i = 0
i < n ?
No
Print final balance, deposits, withdrawals, rejected
Stop
Yes
Input type and amount
amount = float(amount)
amount <= 0 ?
Yes
No
rejected += 1
t.upper() == "D" ?
Yes
balance += amount
deposits += 1
No
t.upper() == "W" ?
Yes
amount <= 3000 and amount <= balance ?
No
No
Yes
rejected += 1
balance -= amount
withdrawals += 1
rejected += 1
i = i + 1

6. Pseudocode

```
START
balance ← 5000
Read n
REPEAT n times
  Read type, amount
  If amount ≤ 0: reject
  Else if type = D: balance ← balance + amount
  Else if type = W and amount ≤ 3000 and amount ≤ balance: balance ← balance - amount
  Else: reject
Display balance and summary
STOP
```

7. Python Source Code

Case - 13.py

```python
# ATM Transaction Sequence Simulation – Easy

balance = 5000
deposits = 0
withdrawals = 0
rejected = 0

n = int(input("Number of transactions: "))

for i in range(n):
    t, amount = input("Enter type and amount: ").split()
    amount = float(amount)

    if amount <= 0:
        rejected += 1
    elif t.upper() == "D":
        balance += amount
        deposits += 1
    elif t.upper() == "W":
        if amount <= 3000 and amount <= balance:
            balance -= amount
            withdrawals += 1
        else:
            rejected += 1
    else:
        rejected += 1

print("Final balance:", balance)
print("Deposits:", deposits)
print("Withdrawals:", withdrawals)
print("Rejected:", rejected)

```

8. Test Cases

Test	Input	Expected Result
1	D 1000	Balance 6000; deposits 1; withdrawals 0; rejected 0
2	W 1200	Balance 3800; deposits 0; withdrawals 1; rejected 0
3	W 4000	Rejected because withdrawal limit is exceeded
4	W 7000	Rejected because balance is insufficient

9. Output Screenshot



10. Conclusion

The ATM Transaction Sequence Simulation program was developed using basic Python constructs from the prescribed syllabus. It accepts the required inputs, applies the stated conditions and calculations, and produces a clear result for valid and invalid cases.

Case Study 2: Student Performance Trend Analysis — Easy

1. Problem Statement

Compare assessment scores across multiple examinations, calculate improvements and declines, identify consistently weak subjects, and classify students according to defined criteria.

2. Objectives

- Compare scores between two assessments.
- Calculate improvement or decline for each subject.
- Identify weak subjects using a supplied threshold.
- Classify overall performance.

3. Assumptions

- Three subjects are used: Python, Maths and English.
- Two examinations are compared.
- A score below 40 is considered weak.
- Overall average of 75 or above is Excellent, 60 or above is Good, 40 or above is Average, otherwise Needs Improvement.

4. Algorithm

1. Read marks for three subjects in Exam 1 and Exam 2.

2. For each subject calculate Exam 2 − Exam 1.

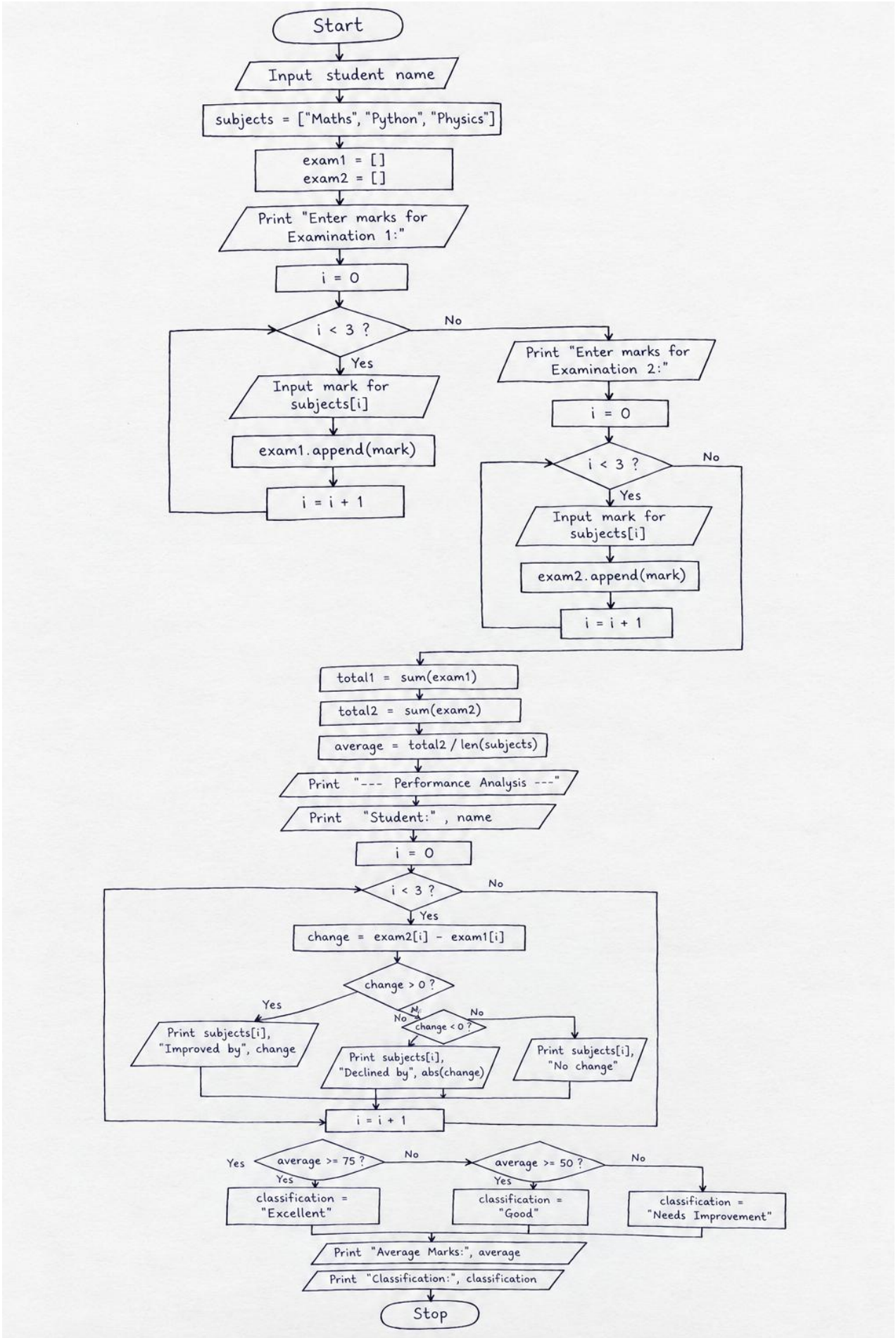
3. Calculate the average of Exam 2.

4. Identify subjects whose Exam 2 mark is below 40.

5. Classify the student from the average.

6. Display the trend and classification.

5. Flowchart


Start
Input student name
subjects = ["Maths", "Python", "Physics"]
exam1 = []
exam2 = []
Print "Enter marks for Examination 1:"
i = 0
i < 3 ?
No
Yes
Input mark for subjects[i]
exam1.append(mark)
i = i + 1
Print "Enter marks for Examination 2:"
i = 0
i < 3 ?
No
Yes
Input mark for subjects[i]
exam2.append(mark)
i = i + 1
total1 = sum(exam1)
total2 = sum(exam2)
average = total2 / len(subjects)
Print "--- Performance Analysis ---"
Print "Student:", name
i = 0
i < 3 ?
No
Yes
change = exam2[i] - exam1[i]
change > 0 ?
Yes
No
change < 0 ?
No
Print subjects[i], "Improved by", change
Print subjects[i], "Declined by", abs(change)
Print subjects[i], "No change"
i = i + 1
average >= 75 ?
Yes
No
classification = "Excellent"
average >= 50 ?
Yes
No
classification = "Good"
classification = "Needs Improvement"
Print "Average Marks:", average
Print "Classification:", classification
Stop

6. Pseudocode

```
START
Read Exam 1 marks
Read Exam 2 marks
For each subject: change ← second − first
Find average of second exam
For each subject, if mark < 40 mark it weak
Classify average
Display results

Stop
```

7. Python Source Code

Case - 17.py

C:\Users\PYCHARM\ABL\Case - 17.py

```python
# Stud

name = input("Enter student name: ")

subjects = ["Maths", "Python", "Physics"]

exam1 = []
exam2 = []

print("Enter marks for Examination 1:")
for subject in subjects:
    mark = int(input(subject + ": "))
    exam1.append(mark)

print("Enter marks for Examination 2:")
for subject in subjects:
    mark = int(input(subject + ": "))
    exam2.append(mark)

total1 = sum(exam1)
total2 = sum(exam2)

average = total2 / len(subjects)

print("\n--- Performance Analysis ---")
print("Student:", name)

for i in range(len(subjects)):
    change = exam2[i] - exam1[i]

    if change > 0:
        print(subjects[i], "Improved by", change)
    elif change < 0:
        print(subjects[i], "Declined by", abs(change))
```

```
        else:
            print(subjects[i], "No change")

if average >= 75:
    classification = "Excellent"
elif average >= 50:
    classification = "Good"
else:
    classification = "Needs Improvement"

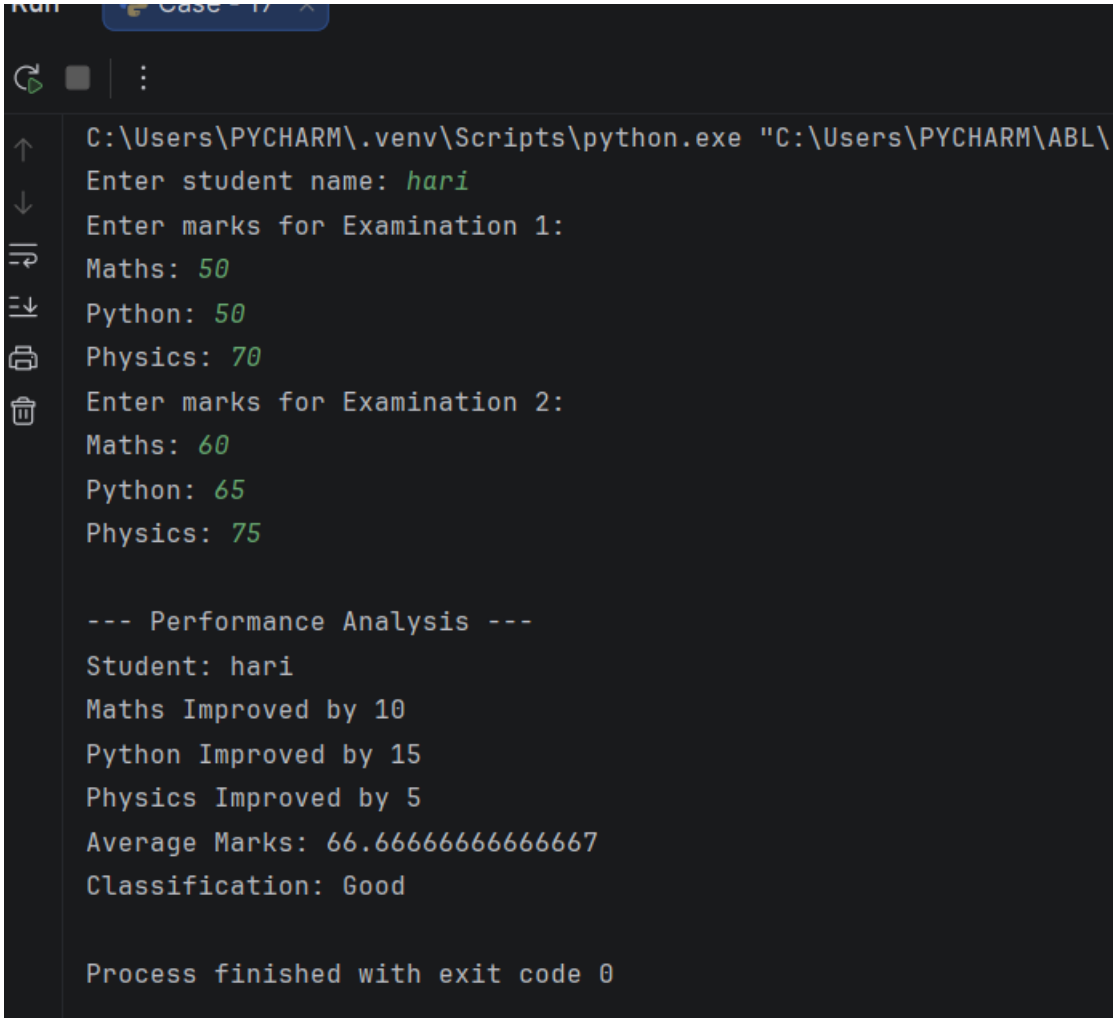
print("Average Marks:", average)
print("Classification:", classification)

```

8. Test Cases

Test	Input	Expected Result
1	50 60 70 / 60 65 75	Changes +10, +5, +5; classification Good
2	80 70 75 / 85 75 80	Changes +5, +5, +5; classification Excellent
3	45 35 50 / 35 38 55	Python and Maths weak; classification Average
4	30 35 25 / 25 30 20	All weak; classification Needs Improvement

9. Output Screenshot



10. Conclusion

The Student Performance Trend Analysis program was developed using basic Python constructs from the prescribed syllabus. It accepts the required inputs, applies the stated conditions and calculations, and produces a clear result for valid and invalid cases.

Case Study 3: Food Waste Monitoring System — Easy

1. Problem Statement

Compare prepared meals, meals served, and leftovers across several days; calculate wastage percentages and identify days exceeding acceptable limits.

2. Objectives

- Calculate leftover meals each day.
- Calculate the wastage percentage.
- Identify days exceeding the permitted wastage limit.
- Summarize total wastage.

3. Assumptions

- Prepared meals are positive integers.
- Wastage percentage = leftovers / prepared × 100.
- A day is excessive when wastage is greater than 10%.

4. Algorithm

1. Read the number of days.

2. For each day, read prepared and served meals.

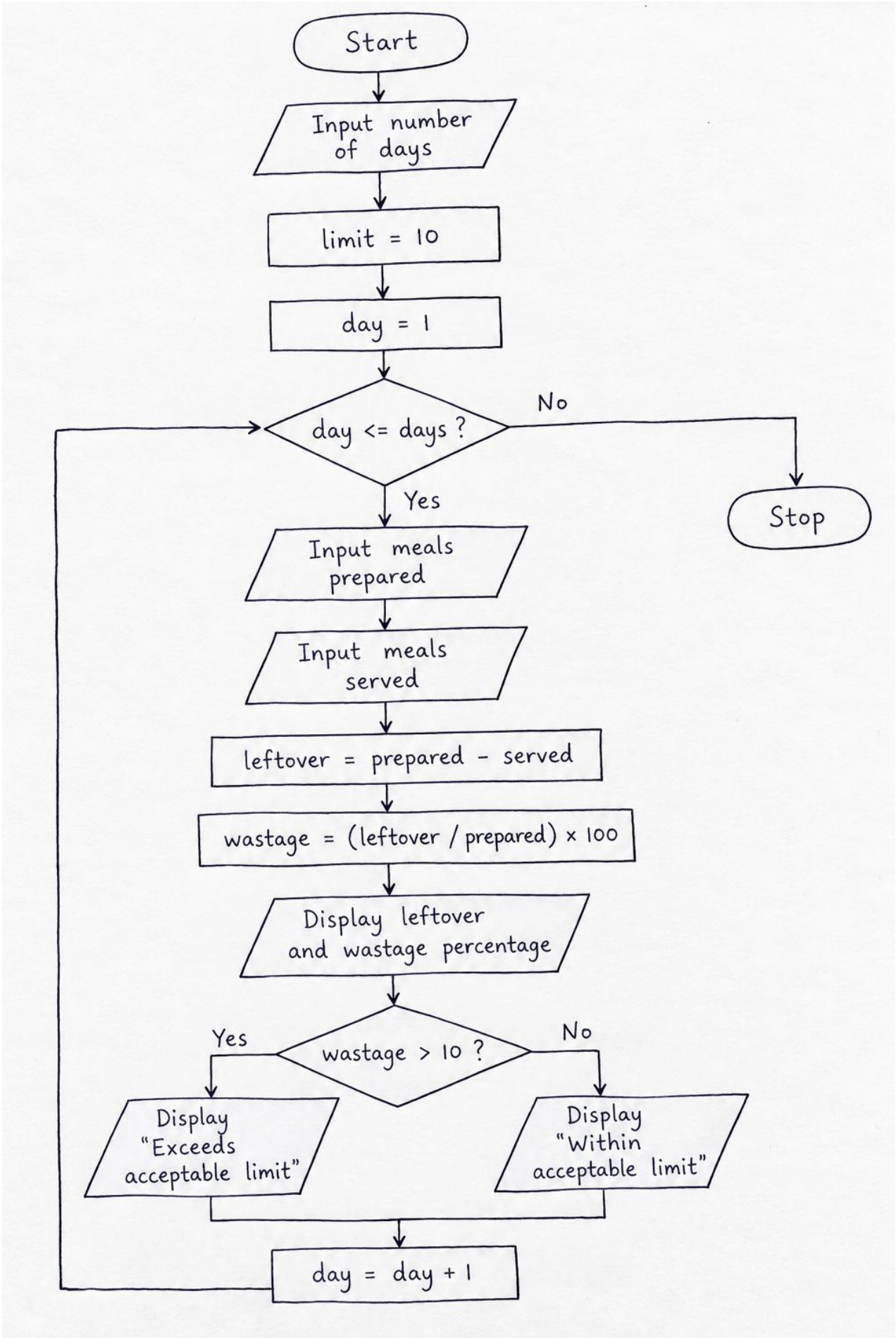
3. Calculate leftovers = prepared − served.

4. Calculate wastage percentage.

5. If percentage > 10, mark the day excessive.

6. Add leftovers to total and display results.

5. Flowchart


Start
Input number of days
limit = 10
day = 1
day <= days ?
No
Yes
Stop
Input meals prepared
Input meals served
leftover = prepared - served
wastage = (leftover / prepared) x 100
Display leftover and wastage percentage
wastage > 10 ?
Yes
No
Display "Exceeds acceptable limit"
Display "Within acceptable limit"
day = day + 1

6. Pseudocode

```
START
Read n
total_waste ← 0
Repeat for each day
  Read prepared, served
  leftover ← prepared − served
  percentage ← leftover/prepared × 100
  If percentage > 10: print excessive
  total_waste ← total_waste + leftover
Print total waste
STOP
```

7. Python Source Code

Case - 53.py

```python
# Food Waste Monitoring System – Easy

days = int(input("Enter number of days: "))

limit = 10

for day in range(1, days + 1):
    print("\nDay", day)

    prepared = int(input("Meals prepared: "))
    served = int(input("Meals served: "))

    leftover = prepared - served
    wastage = (leftover / prepared) * 100

    print("Leftover meals:", leftover)
    print("Wastage percentage:", wastage, "%")

    if wastage > limit:
        print("Exceeds acceptable limit")
    else:
        print("Within acceptable limit")
```

8. Test Cases

Case - 1

Enter number of days: 3

Day 1

Meals prepared: 100

Meals served: 50

Leftover meals: 50

Wastage percentage: 50.0 %

Exceeds acceptable limit

Day 2

Meals prepared: 50

Meals served: 40

Leftover meals: 10

Wastage percentage: 20.0 %

Exceeds acceptable limit

Day 3

Meals prepared: 70

Meals served: 20

Leftover meals: 50

Wastage percentage: 71.42857142857143 %

Exceeds acceptable limit

Case – 2:

Enter number of days: 2

Day 1

Meals prepared: 100

Meals served: 90

Leftover meals: 10

Wastage percentage: 10.0 %

Within acceptable limit

Day 2

Meals prepared: 200

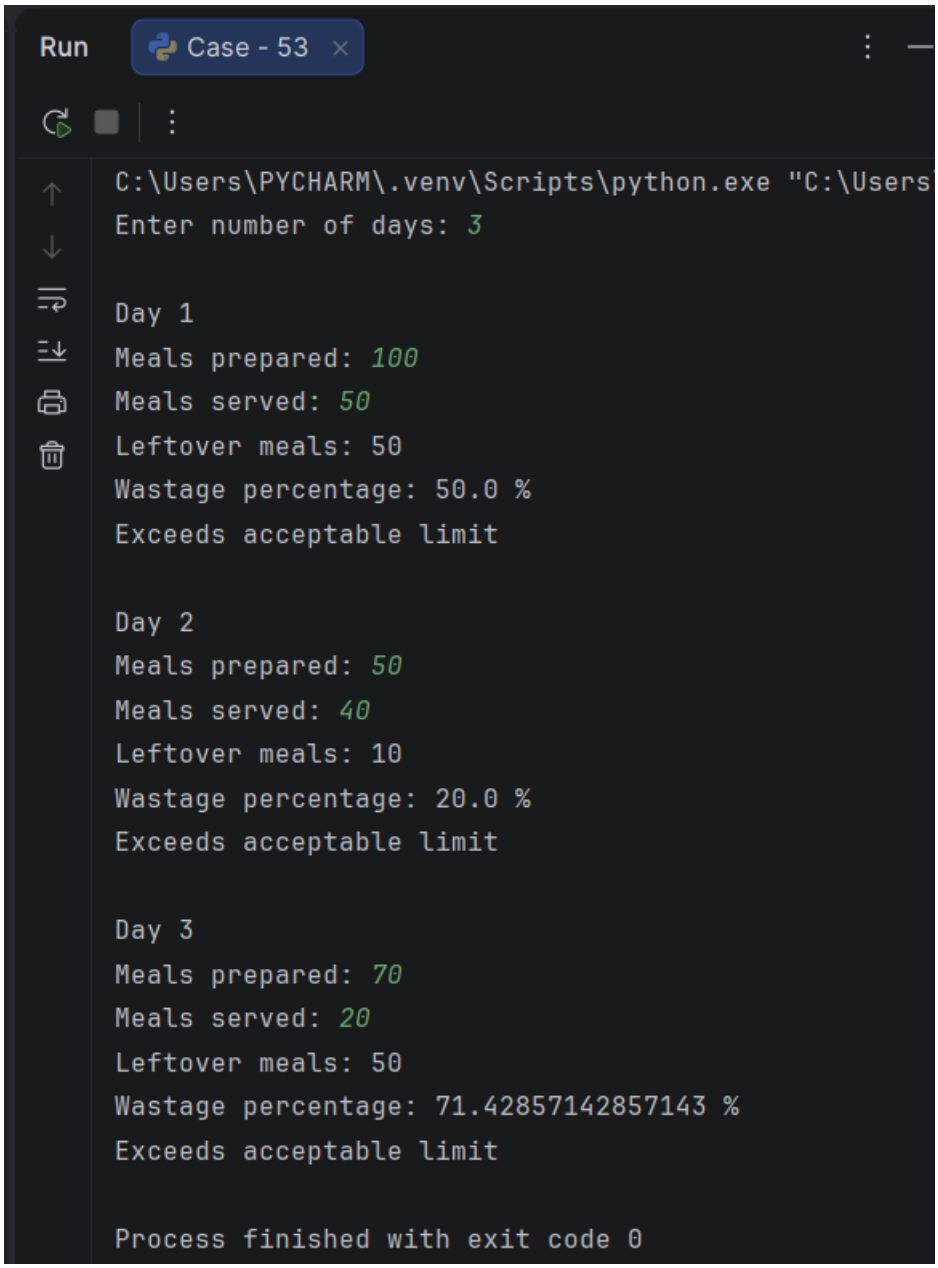
Meals served: 10

Leftover meals: 190

Wastage percentage: 95.0 %

Exceeds acceptable limit

9. Output Screenshot



10. Conclusion

The Food Waste Monitoring System program was developed using basic Python constructs from the prescribed syllabus. It accepts the required inputs, applies the stated conditions and calculations, and produces a clear result for valid and invalid cases.

Case Study 4: Multi-Product Price Fluctuation Analysis — Easy

1. Problem Statement

Process historical product prices stored in lists, calculate percentage changes, identify maximum fluctuations, and flag products exceeding a supplied threshold.

2. Objectives

- Store old and new prices in lists.
- Calculate percentage change for each product.
- Find the largest fluctuation.
- Flag products above the threshold.

3. Assumptions

- All old prices are positive.
- Percentage change uses absolute change divided by old price.
- A fluctuation above 10% is flagged.

4. Algorithm

1. Read product names, old prices and new prices.

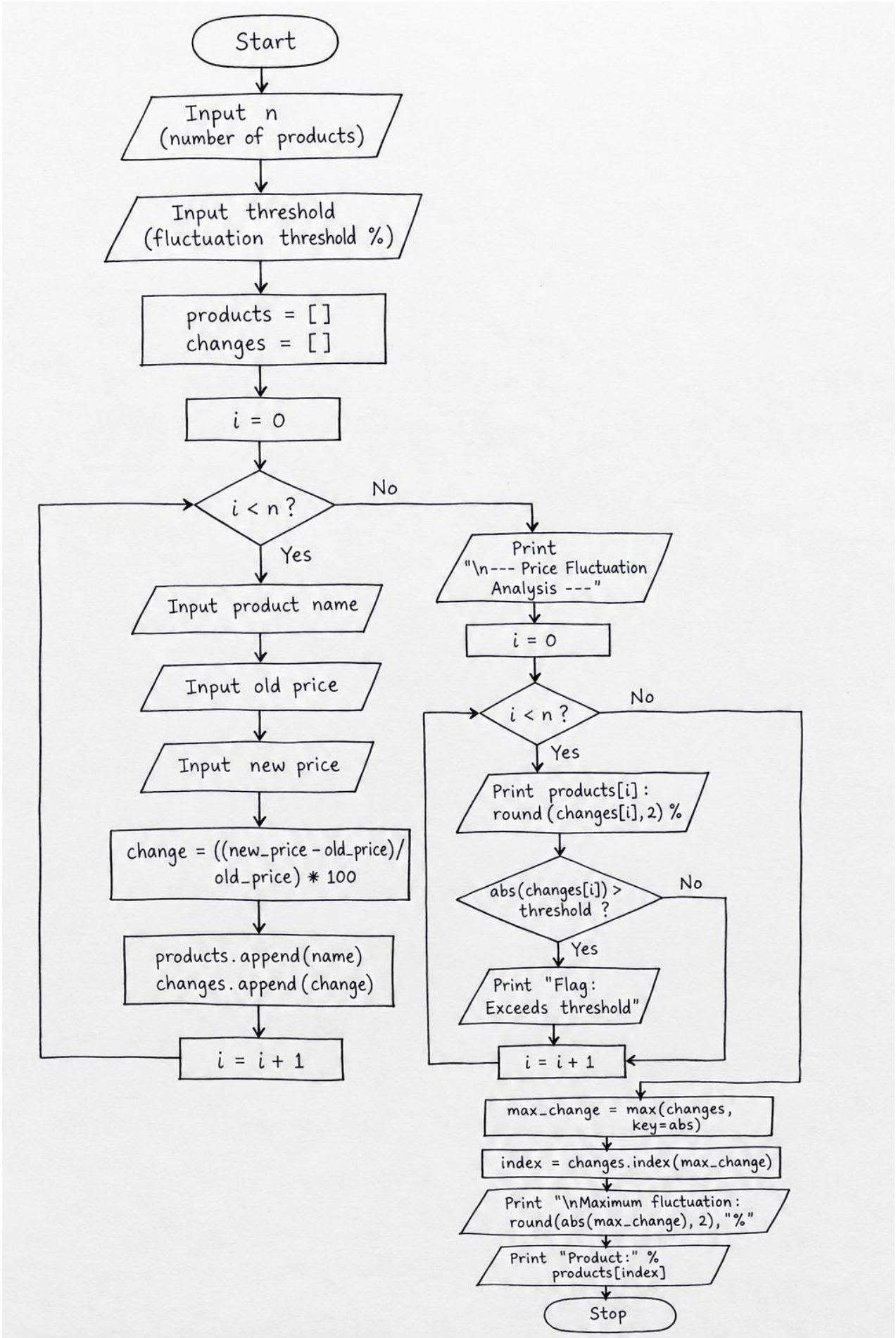
2. Fo each product calculate absolute percentage change.

3. Compare the change with the 10% threshold.

4. Track the largest change and product.

5. Display all results.

5. Flowchart


Start
Input n (number of products)
Input threshold (fluctuation threshold %)
products = []
changes = []
i = 0
i < n ?
No
Yes
Input product name
Input old price
Input new price
change = ((new_price - old_price)/ old_price) * 100
products.append(name)
changes.append(change)
i = i + 1
Print "\n--- Price Fluctuation Analysis ---"
i = 0
i < n ?
No
Yes
Print products[i]: round(changes[i], 2) %
abs(changes[i]) > threshold ?
No
Yes
Print "Flag: Exceeds threshold"
i = i + 1
max_change = max(changes, key=abs)
index = changes.index(max_change)
Print "\nMaximum fluctuation: round(abs(max_change), 2), "%"
Print "Product:" % products[index]
Stop

6. Pseudocode

```
START
Read names, old prices, new prices
max_change ← 0
For each product
  change ← |new − old| / old × 100
  Display change
  If change > 10: flag
  If change > max_change: update maximum
Display maximum
STOP
```

7. Python Source Code

Case - 63.py

```python
n = int(input("Enter number of products: "))
threshold = float(input("Enter fluctuation threshold (%): "))

products = []
changes = []

for i in range(n):
    name = input("Enter product name: ")
    old_price = float(input("Enter old price: "))
    new_price = float(input("Enter new price: "))

    change = ((new_price - old_price) / old_price) * 100

    products.append(name)
    changes.append(change)

print("\n--- Price Fluctuation Analysis ---")

for i in range(n):
    print(products[i], ":", round(changes[i], 2), "%")

    if abs(changes[i]) > threshold:
        print("Flag: Exceeds threshold")

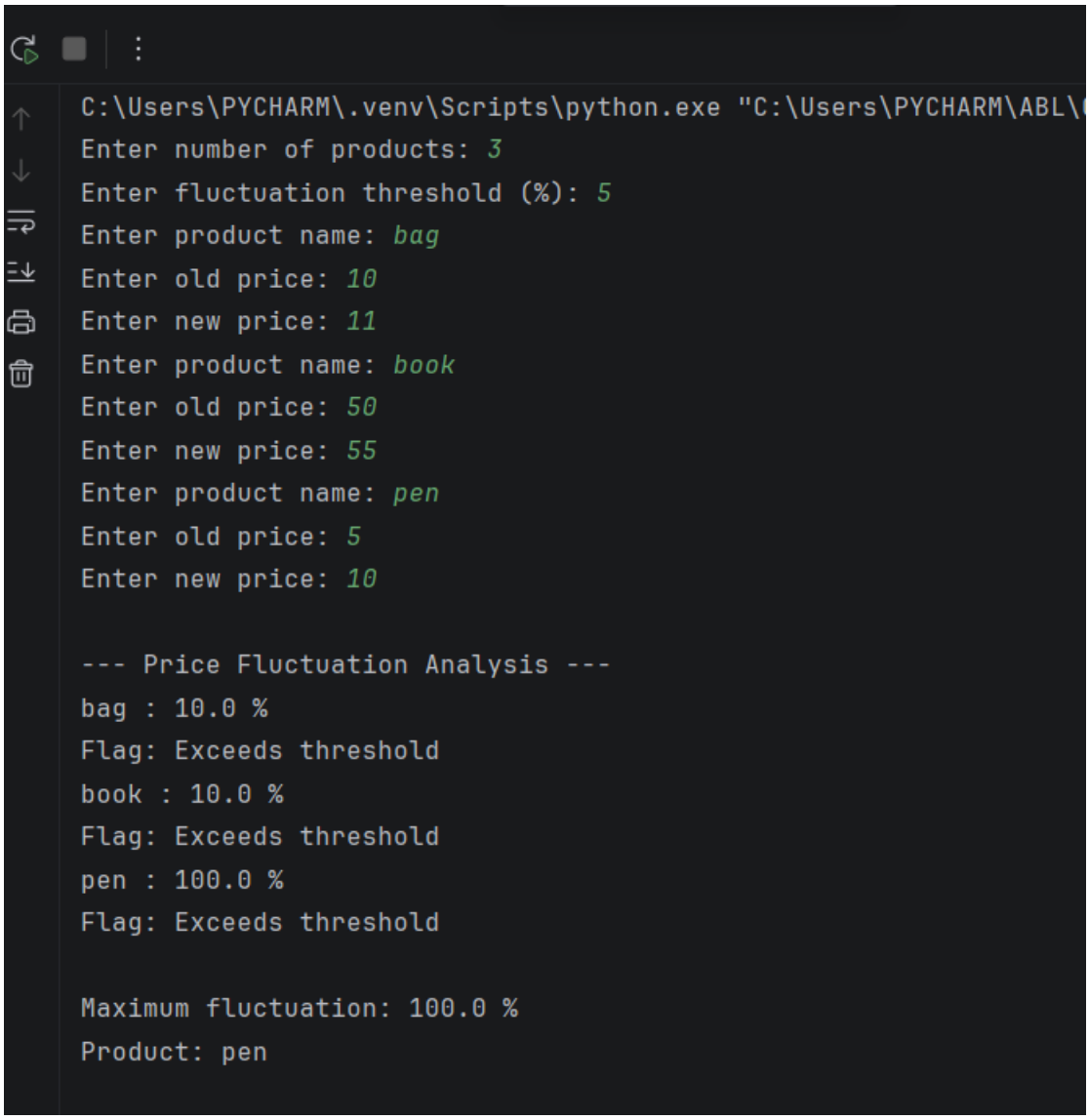
max_change = max(changes, key=abs)
index = changes.index(max_change)

print("\nMaximum fluctuation:", round(abs(max_change), 2), "%")
print("Product:", products[index])
```

8. Test Cases

Test	Input	Expected Result
1	Pen Book Bag / 10 50 100 / 11 55 102	Pen 10%, Book 10%, Bag 2%; maximum 10%
2	A B / 100 200 / 120 180	A 20% flagged; B 10%; maximum A
3	X Y Z / 80 100 50 / 72 120 45	10%, 20% flagged, 10%; maximum Y
4	P / 500 / 600	20% flagged; maximum P

9. Output Screenshot



10. Conclusion

The Multi-Product Price Fluctuation Analysis program was developed using basic Python constructs from the prescribed syllabus. It accepts the required inputs, applies the stated conditions and calculations, and produces a clear result for valid and invalid cases.

Case Study 5: Retail Billing with Multiple Discount Rules — Medium

1. Problem Statement

Process multiple products, quantities, category-specific taxes, threshold discounts, and maximum concession limits to generate an itemized invoice.

2. Objectives

- Calculate item totals.
- Apply category-specific tax rates.
- Apply a threshold discount.
- Respect a maximum discount limit.
- Generate an invoice total.

3. Assumptions

- Electronics tax = 18%, Food tax = 5%, Clothing tax = 12%.
- A 10% discount is applied when subtotal is at least ₹5,000.
- Maximum discount is ₹1,000.
- Product data is entered as name, category, price and quantity.

4. Algorithm

1. Read number of products.

2. For each product calculate price × quantity.

3. Apply the correct tax based on category.

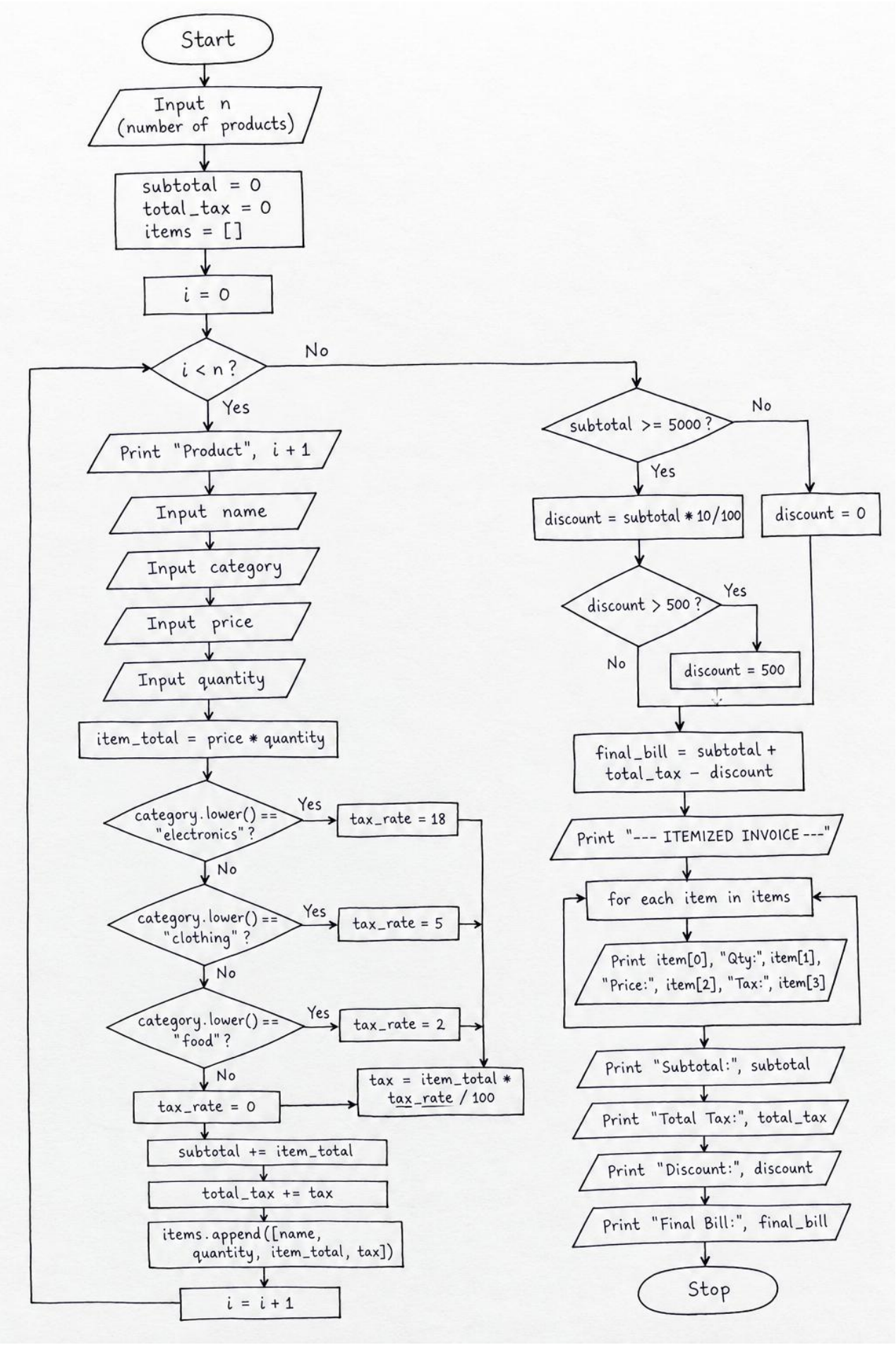
4. Add item totals to subtotal.

5. If subtotal ≥ 5000, calculate 10% discount and cap it at 1000.

6. Add tax and subtract discount.

7. Print the invoice.

5. Flowchart


Start
Input n (number of products)
subtotal = 0
total_tax = 0
items = []
i = 0
i < n ?
No
Yes
Print "Product", i + 1
Input name
Input category
Input price
Input quantity
item_total = price * quantity
category.lower() == "electronics" ?
Yes
tax_rate = 18
No
category.lower() == "clothing" ?
Yes
tax_rate = 5
No
category.lower() == "food" ?
Yes
tax_rate = 2
No
tax_rate = 0
tax = item_total * tax_rate / 100
subtotal += item_total
total_tax += tax
items.append([name, quantity, item_total, tax])
i = i + 1
subtotal >= 5000 ?
No
Yes
discount = subtotal * 10/100
discount = 0
discount > 500 ?
Yes
No
discount = 500
final_bill = subtotal + total_tax - discount
Print "--- ITEMIZED INVOICE ---"
for each item in items
Print item[0], "Qty:", item[1], "Price:", item[2], "Tax:", item[3]
Print "Subtotal:", subtotal
Print "Total Tax:", total_tax
Print "Discount:", discount
Print "Final Bill:", final_bill
Stop

6. Pseudocode

```
START
Read n
subtotal ← 0
tax ← 0
Repeat n times
  Read name, category, price, quantity
  item ← price × quantity
  Add item to subtotal
  Select tax rate from category
  tax ← tax + item × rate
If subtotal ≥ 5000: discount ← min(subtotal × 0.10, 1000)
total ← subtotal + tax − discount
Print invoice
STOP
```

7. Python Source Code

Case - 7.py

```python
# Retail Billing with Multiple Discount Rules – Medium

n = int(input("Enter number of products: "))

subtotal = 0
total_tax = 0
items = []

for i in range(n):
    print("\nProduct", i + 1)

    name = input("Enter product name: ")
    category = input("Enter category: ")
    price = float(input("Enter price: "))
    quantity = int(input("Enter quantity: "))

    item_total = price * quantity

    if category.lower() == "electronics":
        tax_rate = 18
    elif category.lower() == "clothing":
        tax_rate = 5
    elif category.lower() == "food":
        tax_rate = 2
    else:
        tax_rate = 0

    tax = item_total * tax_rate / 100

    subtotal += item_total
    total_tax += tax

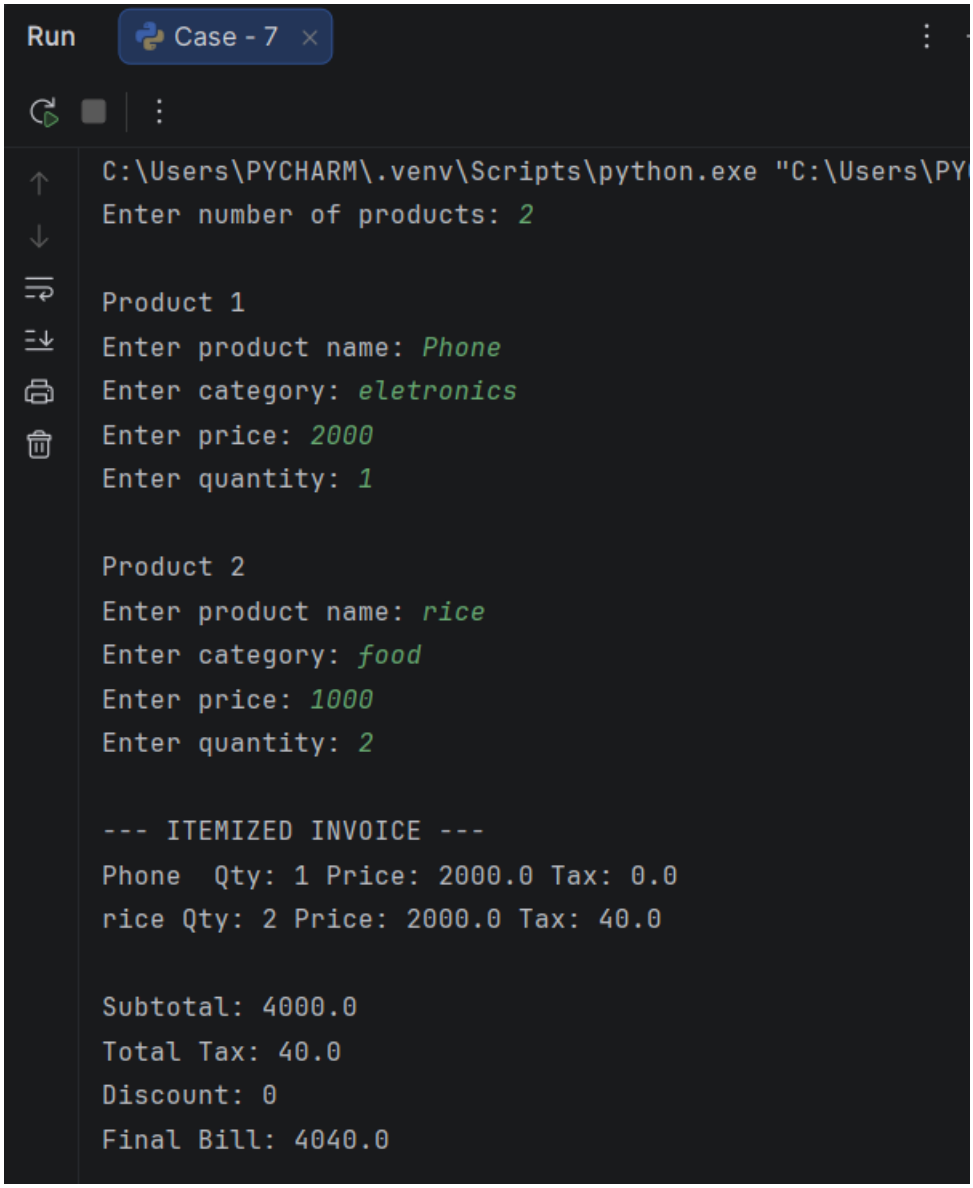
    items.append([name, quantity, item_total, tax])
```

```
34
35      if subtotal >= 5000:
36          discount = subtotal * 10 / 100
37
38          if discount > 500:
39              discount = 500
40      else:
41          discount = 0
42
43      final_bill = subtotal + total_tax - discount
44
45      print("\n--- ITEMIZED INVOICE ---")
46
47      for item in items:
48          print(item[0], "Qty:", item[1],
49                "Price:", item[2],
50                "Tax:", item[3])
51
52      print("\nSubtotal:", subtotal)
53      print("Total Tax:", total_tax)
54      print("Discount:", discount)
55      print("Final Bill:", final_bill)
```

8. Test Cases

Test	Input	Expected Result
1	2 Phone electronics 2000 1 Rice food 1000 2	Subtotal 4000; no discount; tax 280; total 4280
2	2 TV electronics 3000 1 Shirt clothing 2500 1	Subtotal 5500; discount 550; tax 840; total 5790
3	1 Laptop electronics 15000 1	Discount capped at 1000; tax 2700; total 16700
4	2 Book food 100 10 Jeans clothing 500 4	Subtotal 3000; no discount; tax 300; total 3300

9. Output Screenshot



10. Conclusion

The Retail Billing with Multiple Discount Rules program was developed using basic Python constructs from the prescribed syllabus. It accepts the required inputs, applies the stated conditions and calculations, and produces a clear result for valid and invalid cases.

Case Study 6: Fuel Consumption and Route Comparison — Medium

1. Problem Statement

Calculate fuel requirements and journey costs for several vehicles using distance, mileage, fuel prices, and reserve requirements; identify the lowest-cost feasible option.

2. Objectives

- Calculate fuel needed for each vehicle.
- Include reserve fuel.
- Calculate journey cost.
- Select the lowest-cost feasible vehicle.

3. Assumptions

- Fuel required = distance / mileage.
- A 10% reserve is added.
- Each vehicle has a maximum available fuel amount.
- The vehicle is feasible only if required fuel including reserve does not exceed its available fuel.

4. Algorithm

1. Read distance and fuel price.

2. For each vehicle read mileage and available fuel.

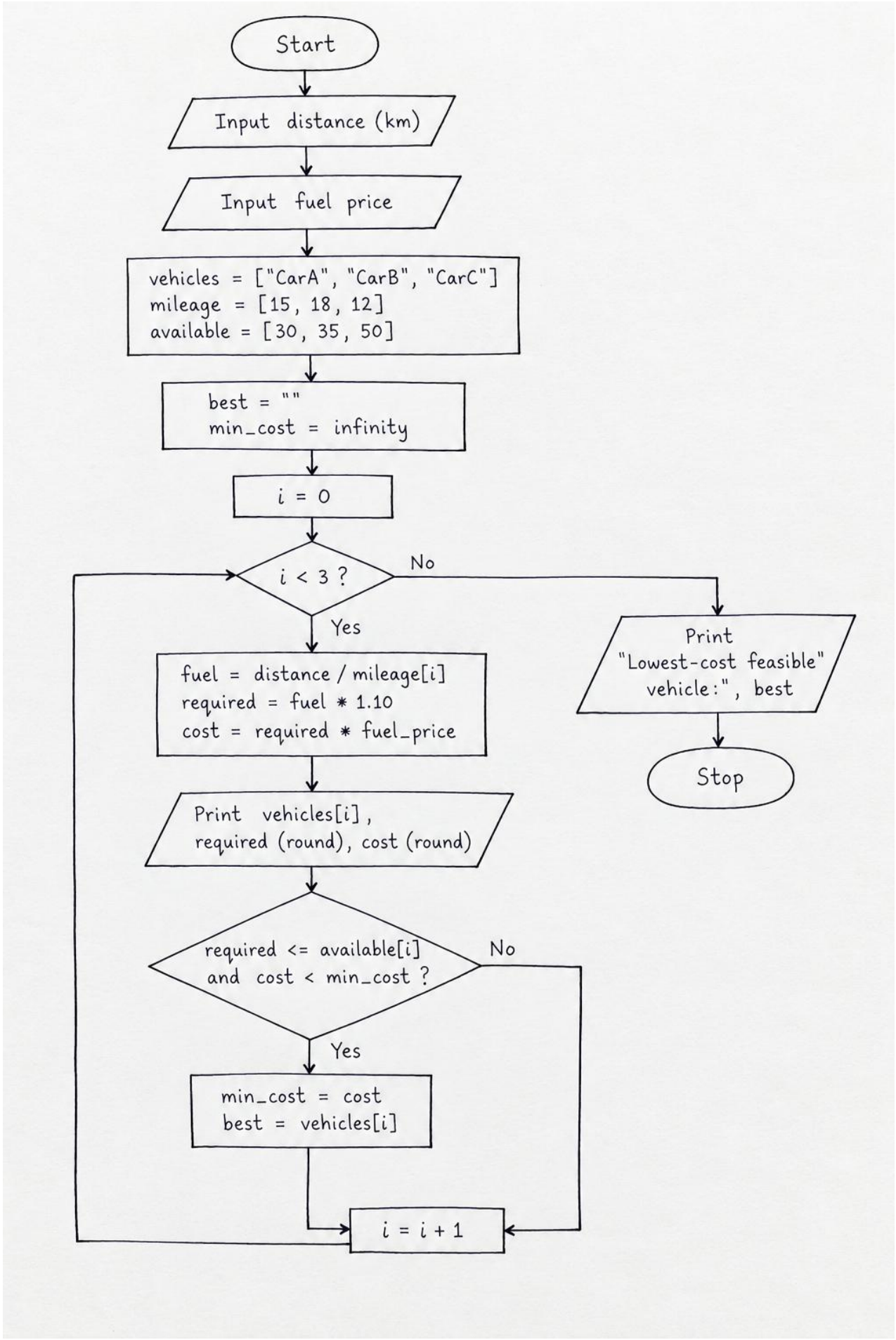
3. Calculate base fuel and reserve-adjusted fuel.

4. Calculate cost.

5. If required fuel is within availability, compare its cost with the current minimum.

6. Display the cheapest feasible vehicle.

5. Flowchart


Start
Input distance (km)
Input fuel price
vehicles = ["CarA", "CarB", "CarC"]
mileage = [15, 18, 12]
available = [30, 35, 50]
best = ""
min_cost = infinity
i = 0
i < 3 ?
No
Yes
Print "Lowest-cost feasible" vehicle:", best
Stop
fuel = distance / mileage[i]
required = fuel * 1.10
cost = required * fuel_price
Print vehicles[i], required (round), cost (round)
required <= available[i] and cost < min_cost ?
No
Yes
min_cost = cost
best = vehicles[i]
i = i + 1

6. Pseudocode

```
START
Read distance, fuel price
For each vehicle
  fuel ← distance/mileage
  required ← fuel × 1.10
  cost ← required × fuel price
  If required ≤ available and cost < minimum: save vehicle
Print cheapest feasible vehicle
STOP
```

7. Python Source Code

Case - 18.py

C:\Users\PYCHARM\ABL\Case - 18.py

```python
# : Fu                                    son – Medium
distance = float(input("Distance (km): "))
fuel_price = float(input("Fuel price: "))

vehicles = ["CarA", "CarB", "CarC"]
mileage = [15, 18, 12]
available = [30, 35, 50]

best = ""
min_cost = float("inf")

for i in range(3):
    fuel = distance / mileage[i]
    required = fuel * 1.10
    cost = required * fuel_price

    print(vehicles[i], "required fuel:", round(required, 2),
          "cost:", round(cost, 2))

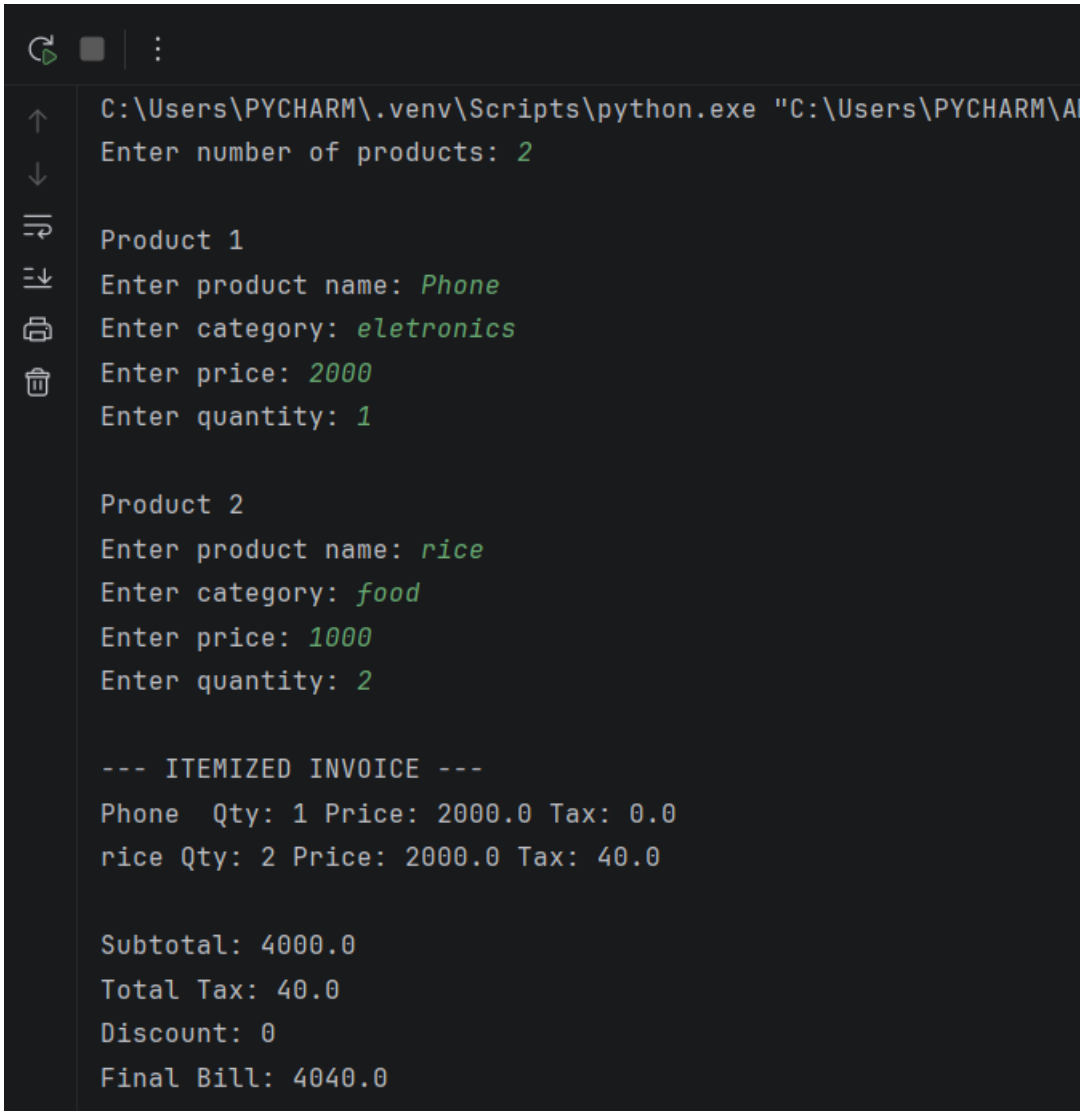
    if required <= available[i] and cost < min_cost:
        min_cost = cost
        best = vehicles[i]

print("Lowest-cost feasible vehicle:", best)
```

8. Test Cases

Test	Input	Expected Result
1	300 km, ₹100/litre	CarA feasible ₹2200; CarB ₹1833.33; CarC ₹2750; best CarB
2	600 km, ₹100/litre	CarA requires 44 L and is not feasible; CarB 36.67 L and not feasible; CarC 55 L not feasible; no feasible vehicle
3	180 km, ₹90/litre	CarB is cheapest feasible
4	120 km, ₹100/litre	CarB is cheapest feasible

9. Output Screenshot



10. Conclusion

The Fuel Consumption and Route Comparison program was developed using basic Python constructs from the prescribed syllabus. It accepts the required inputs, applies the stated conditions and calculations, and produces a clear result for valid and invalid cases.

Case Study 7: Multi-Stage Examination Eligibility System — Medium

1. Problem Statement

Determine eligibility for a final examination using attendance, laboratory completion, internal assessment, and fee-clearance conditions, reporting every unmet requirement.

2. Objectives

- Check each eligibility condition separately.
- Report every unmet condition.
- Declare a student eligible only when all conditions are satisfied.

3. Assumptions

- Attendance must be at least 75%.
- Laboratory completion must be at least 80%.
- Internal assessment must be at least 40/100.
- Fees must be cleared.

4. Algorithm

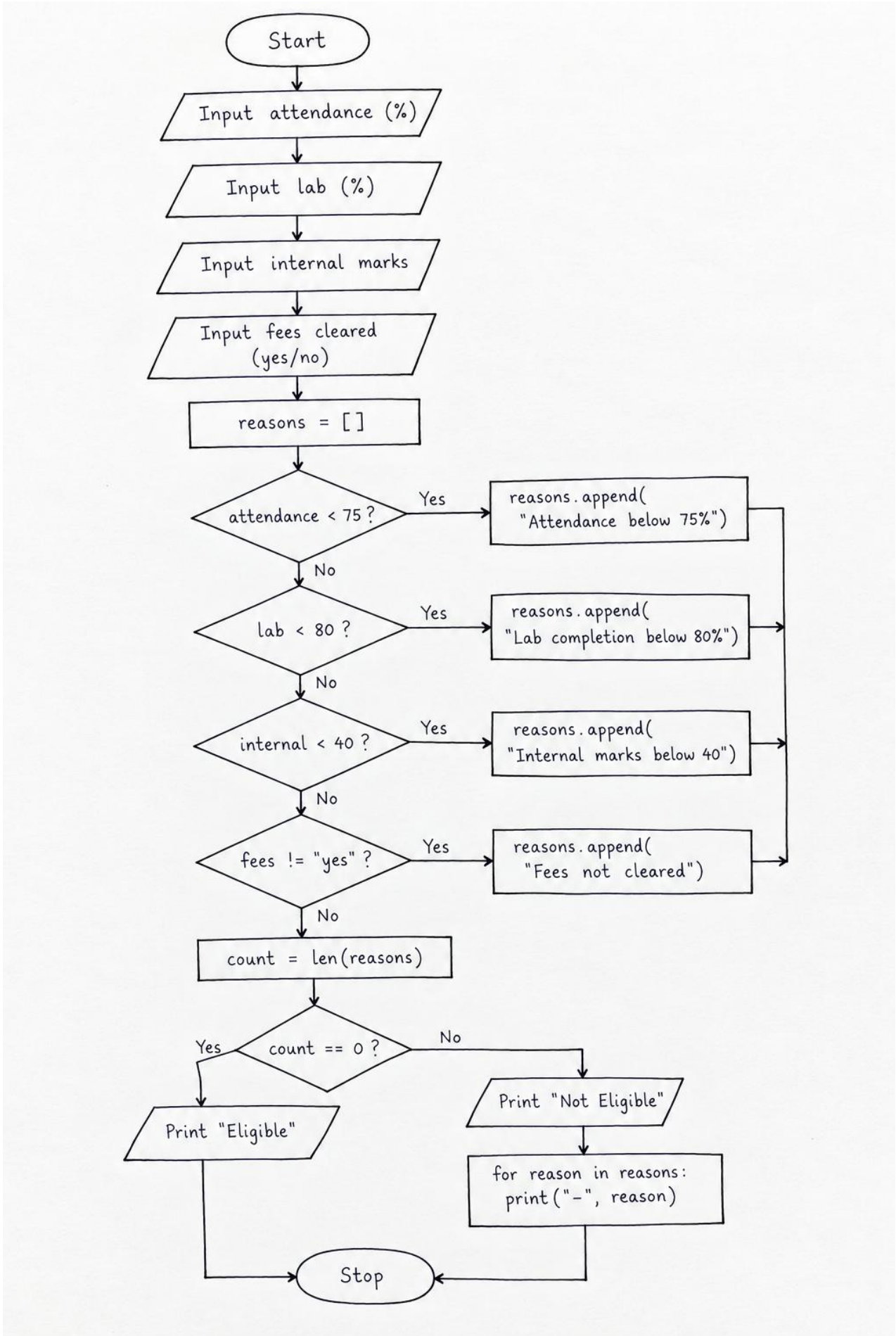
1. Read attendance, lab completion, internal marks and fee status.

2. Create an empty list of unmet requirements.

3. Check each condition and append a reason when it fails.

4. If the list is empty, print Eligible; otherwise print Not Eligible and every reason.

5. Flowchart


Start
Input attendance (%)
Input lab (%)
Input internal marks
Input fees cleared (yes/no)
reasons = []
attendance < 75 ?
Yes
reasons.append("Attendance below 75%")
No
lab < 80 ?
Yes
reasons.append("Lab completion below 80%")
No
internal < 40 ?
Yes
reasons.append("Internal marks below 40")
No
fees != "yes" ?
Yes
reasons.append("Fees not cleared")
No
count = len(reasons)
count == 0 ?
Yes
No
Print "Eligible"
Print "Not Eligible"
for reason in reasons:
print("-", reason)
Stop

6. Pseudocode

```
START
Read attendance, lab, internal, fees
reasons ← empty list
If attendance < 75: add attendance reason
If lab < 80: add lab reason
If internal < 40: add internal reason
If fees not cleared: add fee reason
If reasons empty: Eligible
Else: Not Eligible and display reasons
STOP
```

7. Python Source Code

Case - 39.py

```python
attendance = float(input("Enter attendance percentage: "))
lab = input("Laboratory completed? (yes/no): ")
internal = float(input("Enter internal assessment marks: "))
fees = input("Fees cleared? (yes/no): ")

eligible = True

print("\nUnmet Requirements:")

if attendance < 75:
    print("- Minimum attendance of 75% is required")
    eligible = False

if lab.lower() != "yes":
    print("- Laboratory work must be completed")
    eligible = False

if internal < 40:
    print("- Minimum internal assessment mark is 40")
    eligible = False

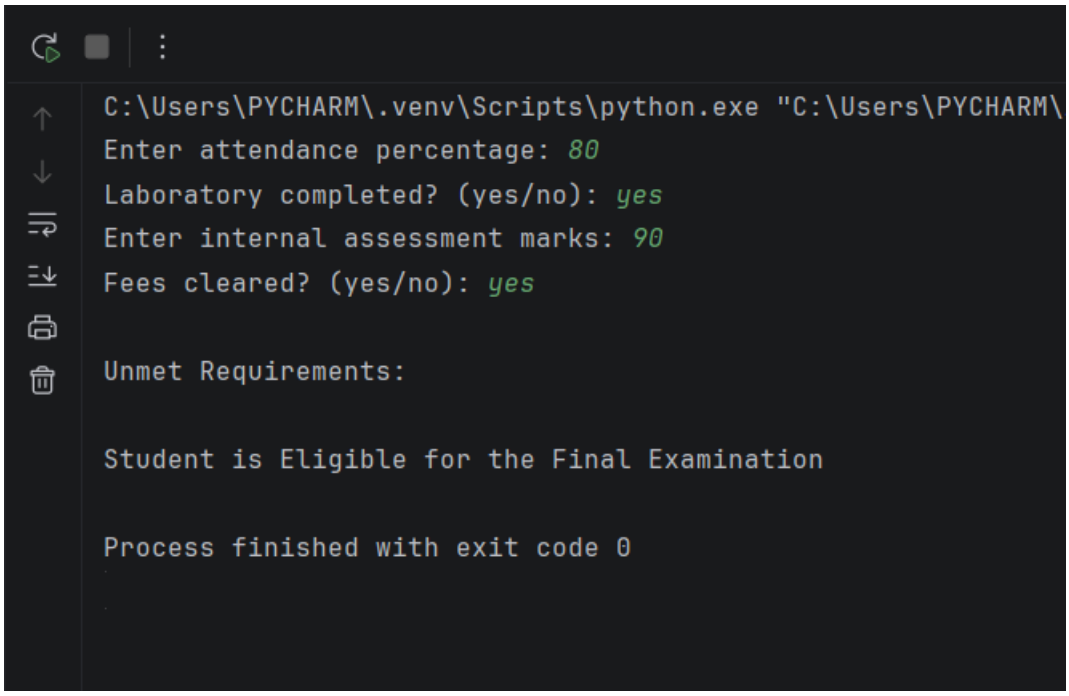
if fees.lower() != "yes":
    print("- Fees must be cleared")
    eligible = False

if eligible:
    print("\nStudent is Eligible for the Final Examination")
else:
    print("\nStudent is Not Eligible for the Final Examination")
```

8. Test Cases

Test	Input	Expected Result
1	80, 90, 60, yes	Eligible
2	70, 90, 60, yes	Not Eligible: attendance
3	80, 70, 30, no	Not Eligible: lab, internal, fees
4	75, 80, 40, yes	Eligible

9. Output Screenshot



10. Conclusion

The Multi-Stage Examination Eligibility System program was developed using basic Python constructs from the prescribed syllabus. It accepts the required inputs, applies the stated conditions and calculations, and produces a clear result for valid and invalid cases.

Case Study 8: Library Borrowing and Fine Reconciliation — Medium

1. Problem Statement

Analyse tuples representing book identifiers, borrowing periods, and return delays; calculate fines, identify overdue records, and reconcile book availability.

2. Objectives

- Store borrowing records as tuples.
- Calculate fines from return delays.
- Identify overdue books.
- Count books currently available after returns.

3. Assumptions

- Each record is (book_id, allowed_days, actual_days).
- Fine is ₹5 for each day beyond the allowed period.
- A record is overdue when actual_days > allowed_days.
- All listed books are returned after processing.

4. Algorithm

1. Read the number of records.

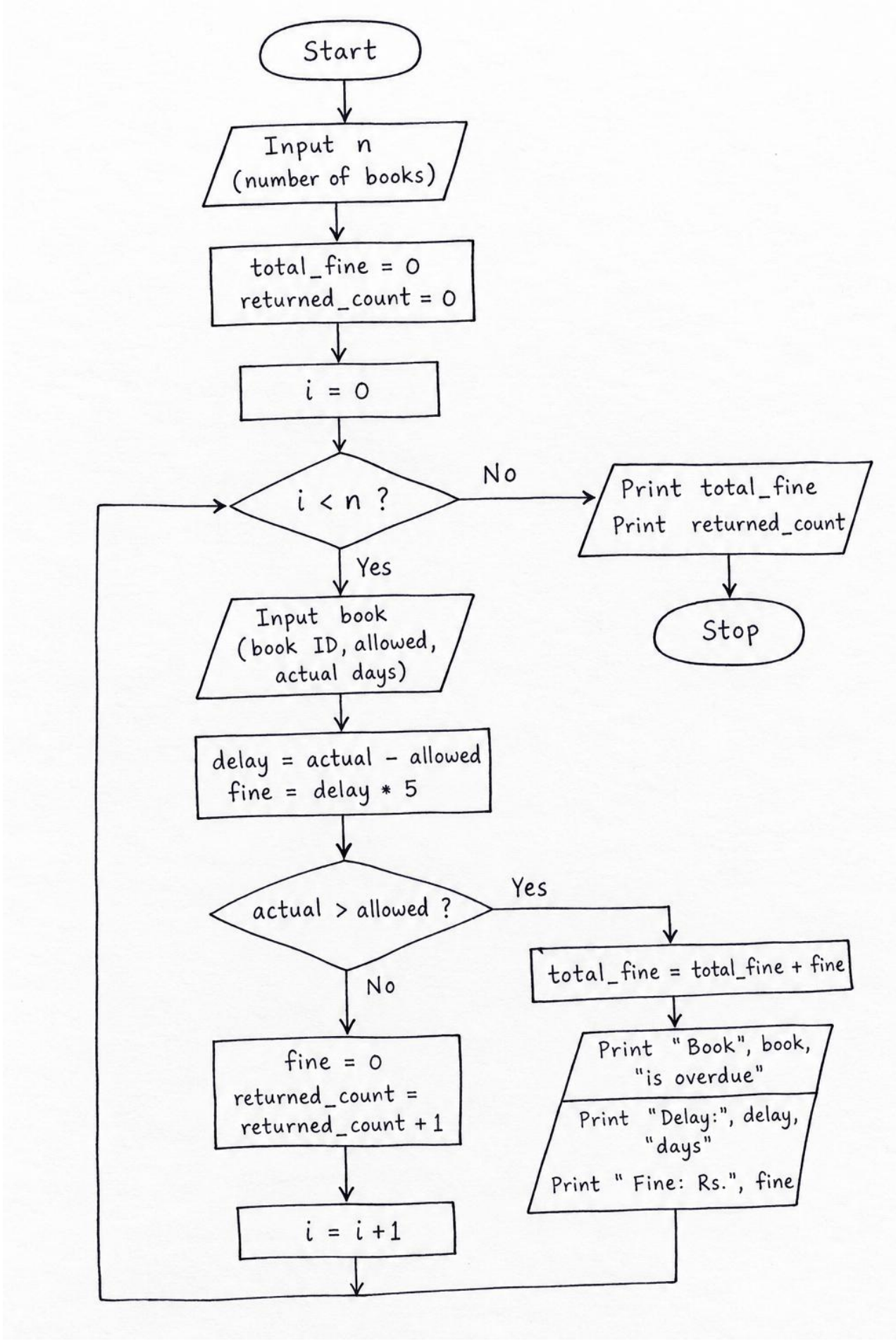
2. For each record create a tuple.

3. Compare actual days with allowed days.

4. Calculate fine as delay × 5 when overdue.

5. Display the fine and count returned books.

5. Flowchart


Start
Input n
(number of books)
total_fine = 0
returned_count = 0
i = 0
i < n ?
No
Print total_fine
Print returned_count
Stop
Yes
Input book
(book ID, allowed,
actual days)
delay = actual - allowed
fine = delay * 5
actual > allowed ?
Yes
total_fine = total_fine + fine
Print "Book", book,
"is overdue"
Print "Delay:", delay,
"days"
Print " Fine: Rs.", fine
No
fine = 0
returned_count =
returned_count + 1
i = i + 1

6. Pseudocode

```
START
Read n
total_fine ← 0
For each record
  Read book, allowed, actual
  If actual > allowed
    delay ← actual − allowed
    fine ← delay × 5
    total_fine ← total_fine + fine
    print overdue
  Else fine ← 0
Print total fine and returned count
STOP
```

7. Python Source Code

Case - 58.py

```python
n = int(input("Enter number of books: "))

total_fine = 0
returned_count = 0

for i in range(n):
    print("\nEnter details for book", i + 1)

    book = input("Enter book ID: ")
    allowed = int(input("Enter allowed borrowing days: "))
    actual = int(input("Enter actual borrowing days: "))

    if actual > allowed:
        delay = actual - allowed
        fine = delay * 5
        total_fine = total_fine + fine

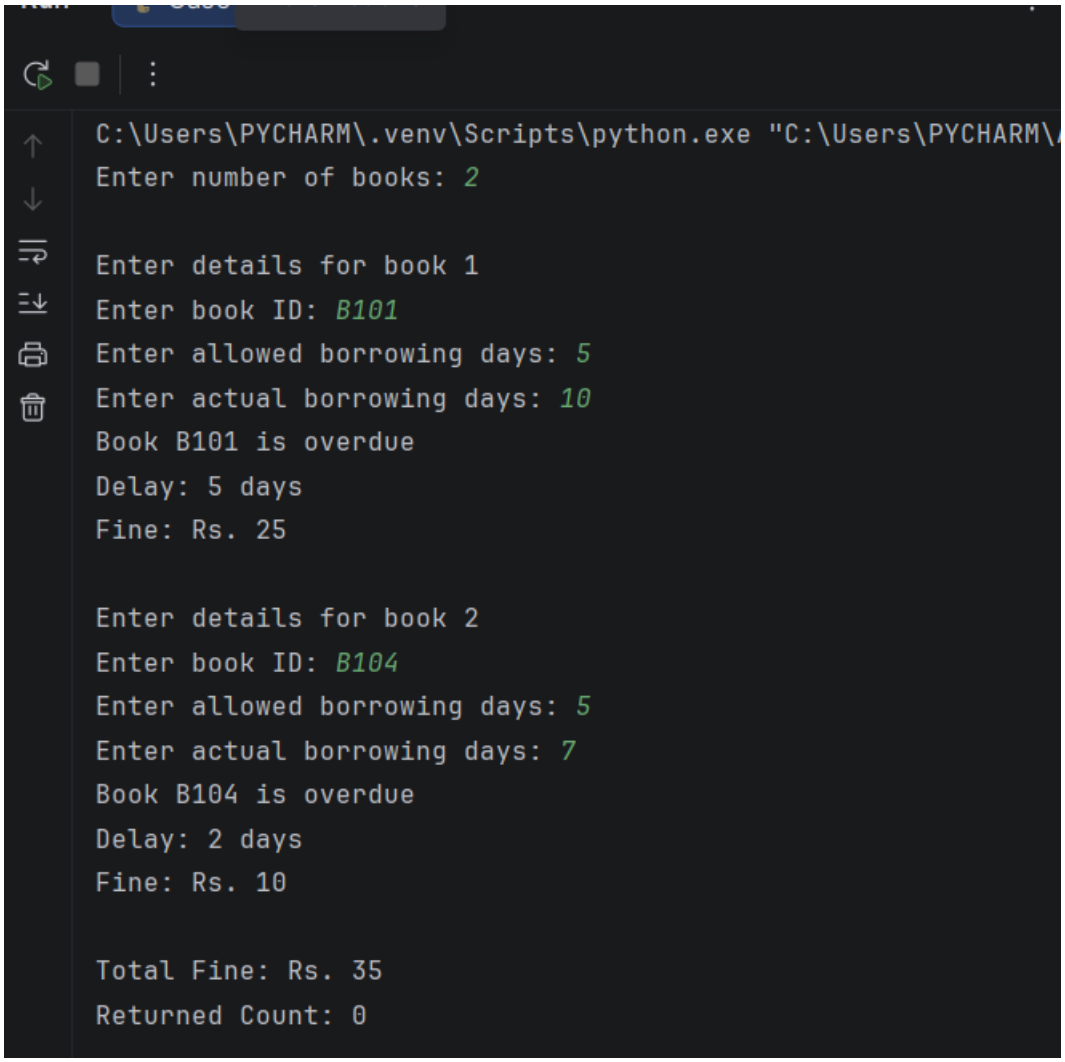
        print("Book", book, "is overdue")
        print("Delay:", delay, "days")
        print("Fine: Rs.", fine)
    else:
        fine = 0
        returned_count = returned_count + 1

print("\nTotal Fine: Rs.", total_fine)
print("Returned Count:", returned_count)
```

8. Test Cases

Test	Input	Expected Result
1	B101 7 10	3 days overdue; fine ₹15
2	B102 14 14	On time; fine ₹0
3	B103 10 15	5 days overdue; fine ₹25
4	B104 5 8	3 days overdue; fine ₹15

9. Output Screenshot



10. Conclusion

The Library Borrowing and Fine Reconciliation program was developed using basic Python constructs from the prescribed syllabus. It accepts the required inputs, applies the stated conditions and calculations, and produces a clear result for valid and invalid cases.

Case Study 9: Student Academic Risk Classification — Hard

1. Problem Statement

Combine lists of marks, attendance, and laboratory scores to classify students into risk categories, identify multiple simultaneous risk factors, and generate a summary.

2. Objectives

- Combine three student-performance measures.
- Detect separate academic risk factors.
- Classify students using multiple conditions.
- Produce a summary of risk categories.

3. Assumptions

- Marks are out of 100, attendance is a percentage, and lab score is out of 100.
- A student has academic risk if marks < 40.
- Attendance risk is attendance < 75%.
- Lab risk is lab score < 40.
- Two or more risk factors means High Risk; one means Medium Risk; none means Low Risk.

4. Algorithm

1. Read names, marks, attendance and lab scores.

2. For each student create an empty risk list.

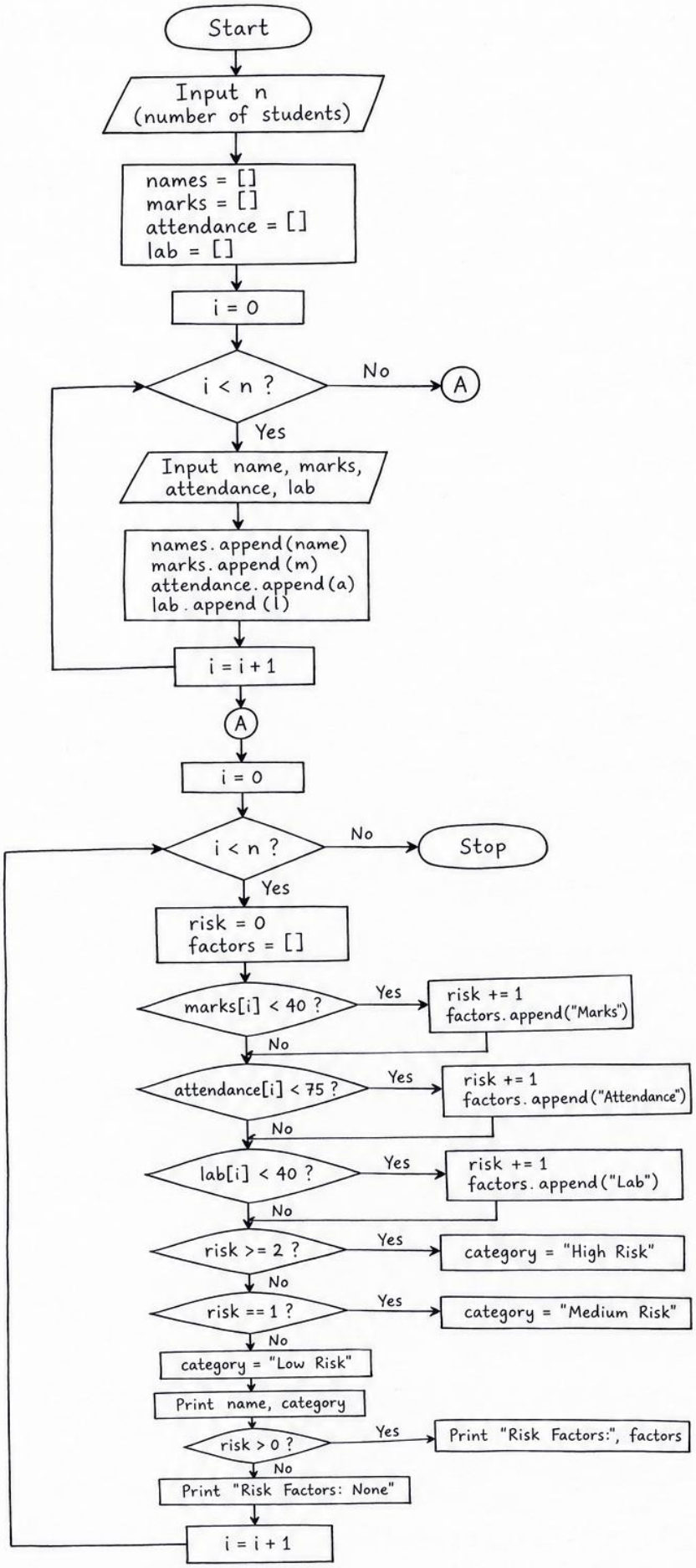
3. Check marks, attendance and lab score conditions.

4. Count risk factors.

5. Classify as High, Medium or Low Risk.

6. Print reasons and maintain category counts.

5. Flowchart


Start
Input n
(number of students)
names = []
marks = []
attendance = []
lab = []
i = 0
i < n ?
No
A
Yes
Input name, marks,
attendance, lab
names.append(name)
marks.append(m)
attendance.append(a)
lab.append(l)
i = i + 1
A
i = 0
i < n ?
No
Stop
Yes
risk = 0
factors = []
marks[i] < 40 ?
Yes
risk += 1
factors.append("Marks")
No
attendance[i] < 75 ?
Yes
risk += 1
factors.append("Attendance")
No
lab[i] < 40 ?
Yes
risk += 1
factors.append("Lab")
No
risk >= 2 ?
Yes
category = "High Risk"
No
risk == 1 ?
Yes
category = "Medium Risk"
No
category = "Low Risk"
Print name, category
risk > 0 ?
Yes
Print "Risk Factors:", factors
No
Print "Risk Factors: None"
i = i + 1

6. Pseudocode

```
START
Read names, marks, attendance, lab scores
Set High, Medium, Low counts to 0
For each student
  risks ← empty list
  If marks < 40: add marks
  If attendance < 75: add attendance
  If lab < 40: add lab
  If number of risks ≥ 2: High Risk
  Else if number = 1: Medium Risk
  Else: Low Risk
  Update count
Display summary
STOP
```

7. Python Source Code

Case - 66.py

```python
#Student Academic Risk Classification – Hard

n = int(input("Enter number of students: "))

names = []
marks = []
attendance = []
lab = []

for i in range(n):
    print("\nStudent", i + 1)

    name = input("Enter name: ")
    m = int(input("Enter marks: "))
    a = float(input("Enter attendance: "))
    l = int(input("Enter lab score: "))

    names.append(name)
    marks.append(m)
    attendance.append(a)
    lab.append(l)

for i in range(n):
    risk = 0
    factors = []

    if marks[i] < 40:
        risk += 1
        factors.append("Marks")

    if attendance[i] < 75:
        risk += 1
        factors.append("Attendance")

```

```
    if lab[i] < 40:
        risk += 1
        factors.append("Lab")

    if risk >= 2:
        category = "High Risk"
    elif risk == 1:
        category = "Medium Risk"
    else:
        category = "Low Risk"

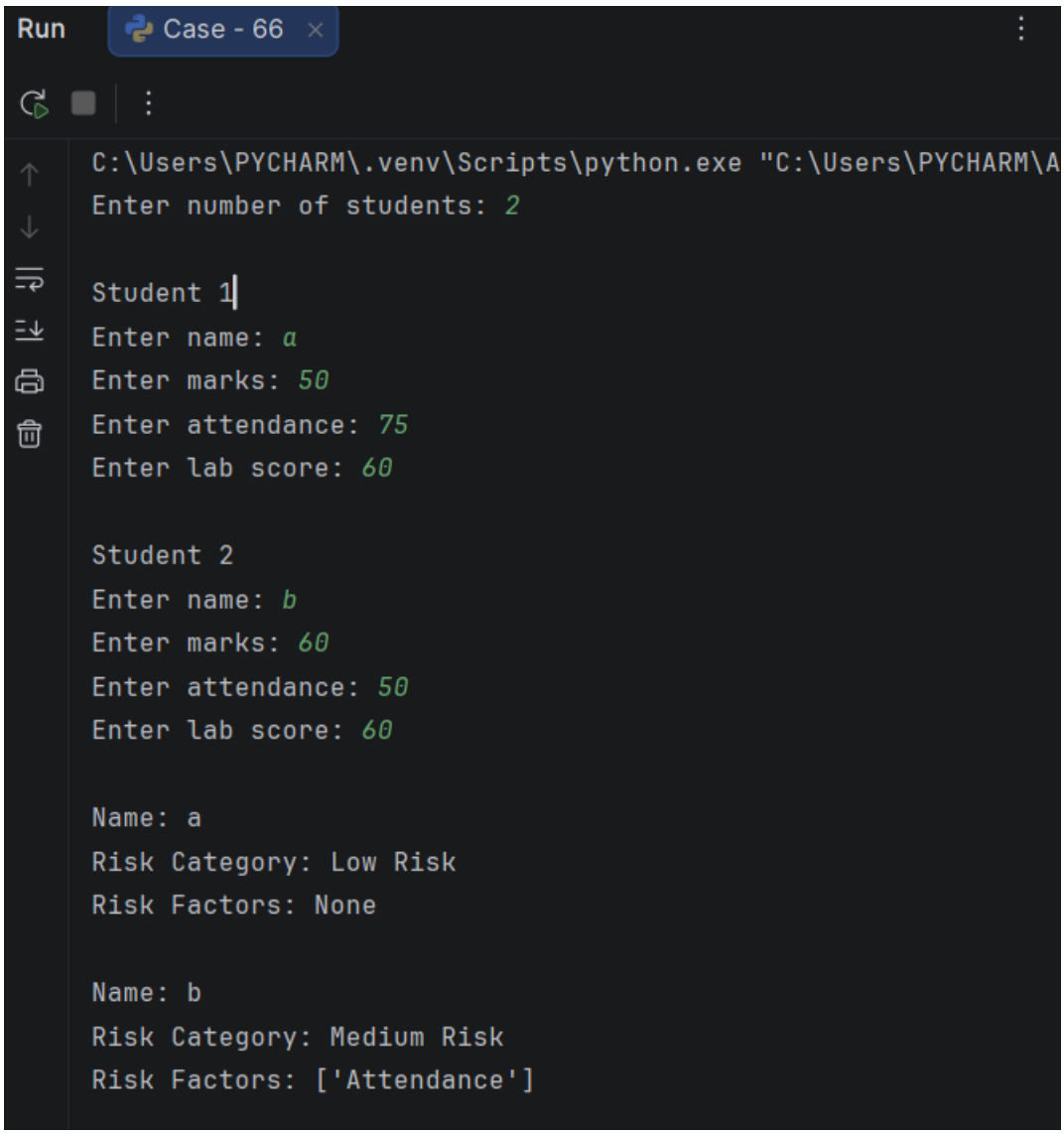
    print("\nName:", names[i])
    print("Risk Category:", category)

    if risk > 0:
        print("Risk Factors:", factors)
    else:
        print("Risk Factors: None")
```

8. Test Cases

Test	Input	Expected Result
1	A B C / 80 35 60 / 90 80 70 / 70 30 50	A Low; B Medium (marks); C Low
2	A B / 30 30 / 60 70 / 30 80	A High (marks, attendance, lab); B Medium (marks)
3	A B C / 39 40 41 / 74 75 76 / 39 40 41	A High; B Low; C Low
4	A B / 20 90 / 60 80 / 20 90	A High; B Low

9. Output Screenshot



10. Conclusion

The Student Academic Risk Classification program was developed using basic Python constructs from the prescribed syllabus. It accepts the required inputs, applies the stated conditions and calculations, and produces a clear result for valid and invalid cases.

Case Study 10: Integrated Student Record Consistency Checker — Hard

1. Problem Statement

Validate student identifiers, names, marks, attendance, and course records stored in lists and tuples; detect duplicates, missing values, and inconsistent academic records.

2. Objectives

- Validate student IDs and required fields.
- Detect duplicate identifiers.
- Check mark and attendance ranges.
- Detect missing course records.
- Generate a reason-wise consistency report.

3. Assumptions

- Student ID must start with S and contain four digits.
- Marks must be between 0 and 100.
- Attendance must be between 0 and 100.
- Each student must have at least one course code.
- Duplicate IDs are invalid.

4. Algorithm

1. Read student records as tuples.

2. For each record validate the ID format.

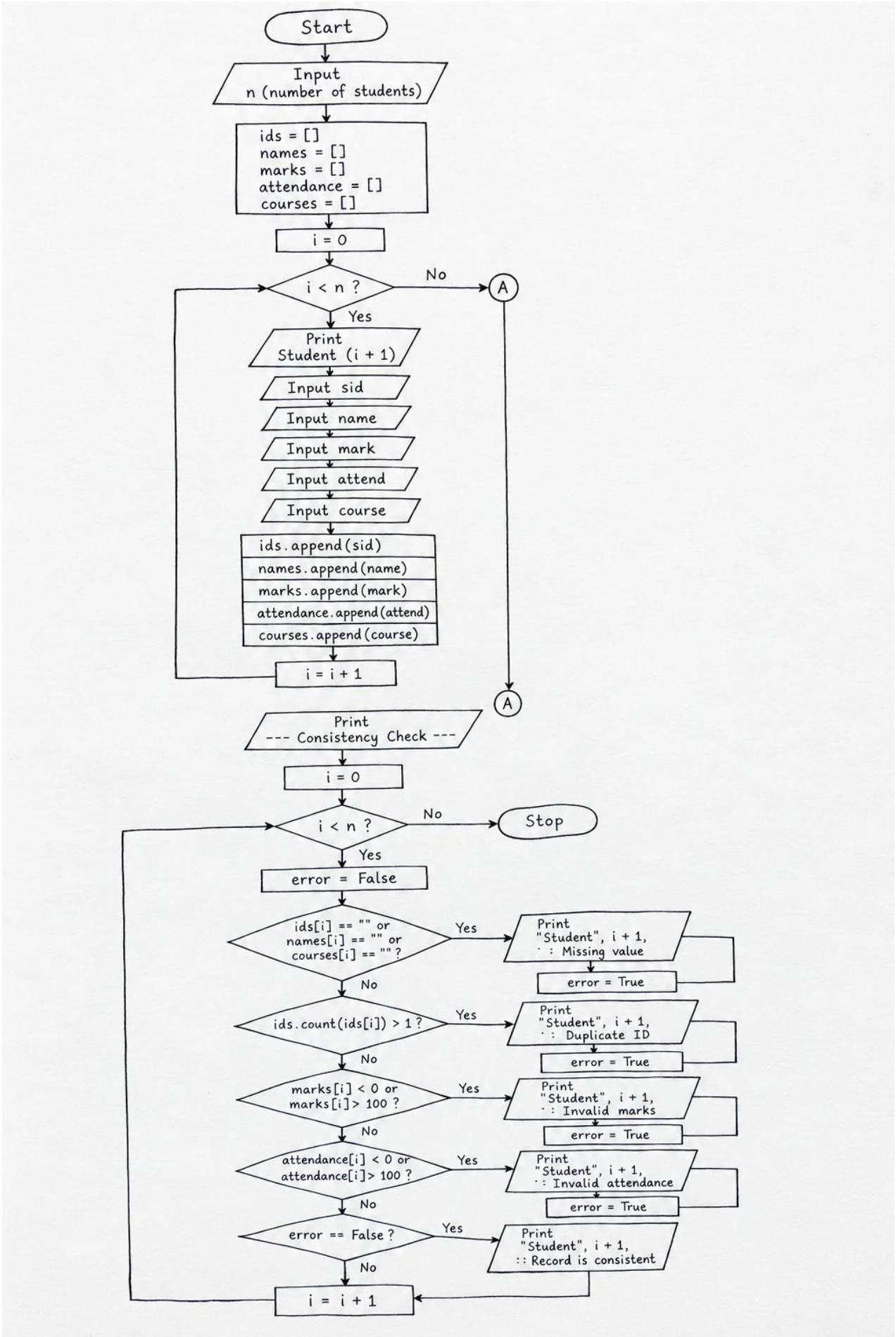
3. Check for duplicate IDs using a list of seen IDs.

4. Check missing name, invalid marks and invalid attendance.

5. Check that at least one course is present.

6. If no problems exist, mark the record consistent; otherwise print all problems.

5. Flowchart


Start
Input
n (number of students)
ids = []
names = []
marks = []
attendance = []
courses = []
i = 0
i < n ?
No
Yes
A
Print
Student (i + 1)
Input sid
Input name
Input mark
Input attend
Input course
ids.append(sid)
names.append(name)
marks.append(mark)
attendance.append(attend)
courses.append(course)
i = i + 1
A
Print
--- Consistency Check ---
i = 0
i < n ?
No
Stop
Yes
error = False
ids[i] == "" or
names[i] == "" or
courses[i] == "" ?
Yes
Print
"Student", i + 1,
": Missing value
error = True
No
ids.count(ids[i]) > 1 ?
Yes
Print
"Student", i + 1,
": Duplicate ID
error = True
No
marks[i] < 0 or
marks[i] > 100 ?
Yes
Print
"Student", i + 1,
": Invalid marks
error = True
No
attendance[i] < 0 or
attendance[i] > 100 ?
Yes
Print
"Student", i + 1,
": Invalid attendance
error = True
No
error == False ?
Yes
Print
"Student", i + 1,
": Record is consistent
No
i = i + 1

6. Pseudocode

```
START
Read n
seen ← empty list
Repeat n times
  Read id, name, marks, attendance, courses
  errors ← empty list
  Validate id structure
  If id already in seen: add duplicate error
  Check missing name
  Check marks and attendance ranges
  Check courses not empty
  Add id to seen
  If errors empty: Consistent
  Else: Inconsistent and display errors
STOP
```

7. Python Source Code

Case - 10.py

Close. Alt+Click to Close Others (Ctrl+F4)

```python
#Integra                                    ker – Hard

n = int(input("Enter number of students: "))

ids = []
names = []
marks = []
attendance = []
courses = []

for i in range(n):
    print("\nStudent", i + 1)

    sid = input("Enter student ID: ")
    name = input("Enter name: ")
    mark = float(input("Enter marks: "))
    attend = float(input("Enter attendance: "))
    course = input("Enter course: ")

    ids.append(sid)
    names.append(name)
    marks.append(mark)
    attendance.append(attend)
    courses.append(course)

print("\n--- Consistency Check ---")

for i in range(n):
    error = False

    if ids[i] == "" or names[i] == "" or courses[i] == "":
        print("Student", i + 1, ": Missing value")
        error = True
```

```
    if ids.count(ids[i]) > 1:
        print("Student", i + 1, ": Duplicate ID")
        error = True

    if marks[i] < 0 or marks[i] > 100:
        print("Student", i + 1, ": Invalid marks")
        error = True

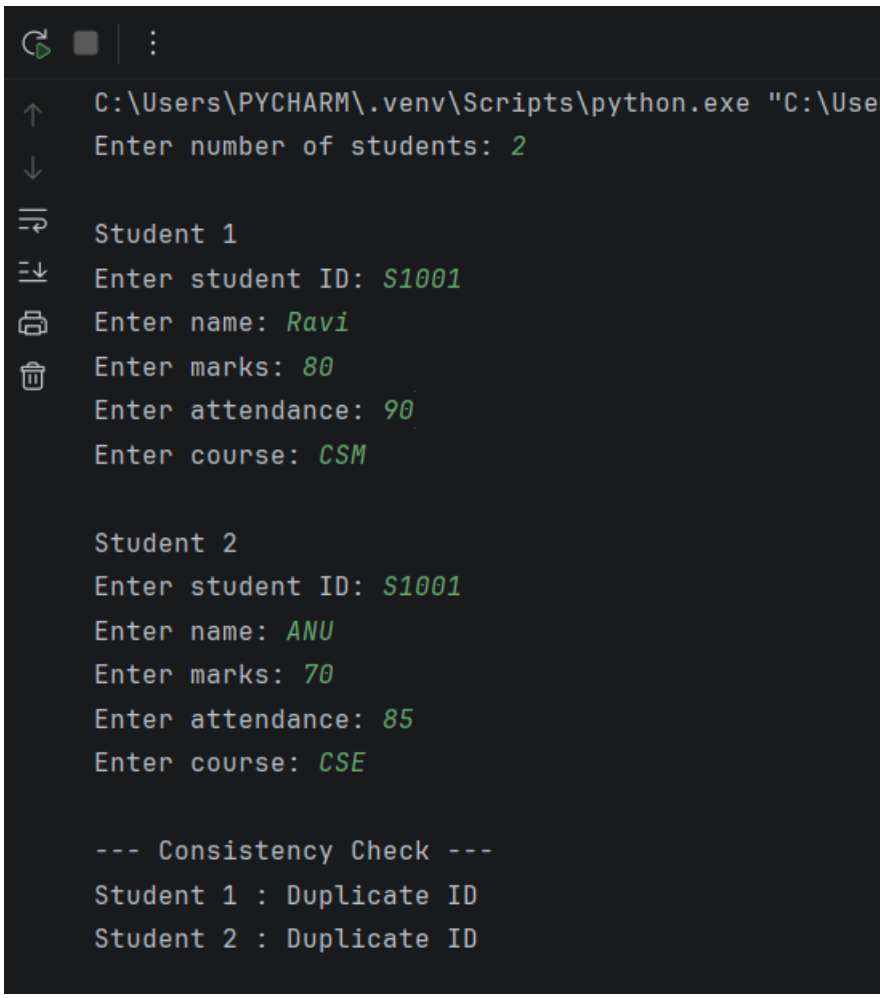
    if attendance[i] < 0 or attendance[i] > 100:
        print("Student", i + 1, ": Invalid attendance")
        error = True

    if error == False:
        print("Student", i + 1, ": Record is consistent")
```

8. Test Cases

Test	Input	Expected Result
1	S1001 Ravi 80 90 CSM	Consistent
2	S1001 Ravi 80 90 CSM; S1001 Anu 70 85 CSE	Second record invalid due to duplicate ID
3	X1234 Ravi 80 90 CSM	Invalid ID
4	S1002 - 120 110 -	Missing name, invalid marks, invalid attendance, missing course

9. Output Screenshot



10. Conclusion

The Integrated Student Record Consistency Checker program was developed using basic Python constructs from the prescribed syllabus. It accepts the required inputs, applies the stated conditions and calculations, and produces a clear result for valid and invalid cases.